# ARC-AGI Agent for Knowledge-Based AI

Sai Suchir Tyada
styada3@gatech.edu

***Abstract*—**This report details the design and evaluation of an ARC-AGI agent developed for the Knowledge-Based AI course. The agent was engineered to solve a wide variety of visual reasoning tasks from the ARC benchmark using a hybrid strategy of algorithmic transformations, object-level reasoning, and learned heuristics. By analyzing input-output training pairs, the agent dynamically constructs a ranked set of candidate transformations and applies them to the test input using a probabilistic selection mechanism. Its architecture combines short-term perception of input grids with long-term knowledge representation accumulated during training, mirroring aspects of human-like cognition. Performance evaluation across 96 problems demonstrated the agent's strengths in segmentation-based and global transformation tasks, while also highlighting challenges in handling ambiguity and compositional complexity. This report presents the agent's underlying mechanisms, experimental outcomes, and a cognitive-level comparison to human reasoning strategies.

## 1 AGENT DESCRIPTION

The ARC-AGI agent developed for this project is a modular, rule-based system designed to address a curated list of ARC problems from the ARC benchmark Chollet, 2019. At its core, the agent uses a curated set of algorithmic transformations, combined with structural heuristics and object-level segmentation logic, to get as close as possible to human-like reasoning over a NumPy array input data Harris et al., 2020. Rather than hard-coding a specific rule per problem, the agent follows a flexible interpretation pipeline that generalizes across tasks by identifying shared transformation patterns and spatial structures.

Upon receiving an ARC problem, the agent first parses the training input-output examples to extract key features: shape, color composition, object boundaries, and spatial layout. These observations form a type of short-term knowledge that guides candidate solution strategies. Each input is compared to its corresponding output to infer which transformations may be applicable. These include geometric op-

erations (i.e. rotation, reflection), logical bitwise operators (i.e. XOR, OR, NAND, AND), color remapping, object duplication, or spatial masking.

### 1.1 Decision-making Process

The decision-making process is orchestrated by the `determine_transformations` method. In this component, the agent applies a "try-and-test" approach: it systematically applies each transformation in its library to the training input and verifies whether the result matches the known output. This process is run in two stages, first we start by processing whole shapes, then we shift to an object detection-based approach wherein we run the same transformations on smaller sub-clusters of the NumPy array. Successful matches from this process are recorded, and the frequency of success is used to build a probability distribution over all attempted transformations. This distribution makes up a form of long-term knowledge/memory of which strategies tend to succeed across the given Arc problem's training samples.

Once training-time transformations are ranked and sorted, the agent transitions to inference. It applies the top-ranked transformations to the test input and generates up to three predictions. This prioritization balances exploration with confidence, allowing the agent to propose high-likelihood candidates without overfitting to a single heuristic. This also allows for the decision-making process to propose lower probability transformations as well. It was noticed that in some edge cases 2 transformations seemed to fit well to the given training set but only 1 was correct. This allows for the agent to check those transformations as well. When processing test inputs, the agent conditionally applies object-level reasoning using the `ndimage` package from SciPy library Virtanen et al., 2020. If the input grid is structurally divisible (such as containing distinct objects or sections) the agent segments the array and applies transformations independently to each region before recombining the results.

Furthermore, several functions also contribute to perception and analysis. Object detection is implemented via connected component labeling, allowing the agent to isolate and individually process elements of the grid. Custom enclosure detection was implemented to identify whether shapes are hollow or filled, enabling the agent to apply interior fill rules when necessary. Color logic is handled through frequency analysis and mapping tables, ensuring consistency when transforming input to output colors.

Taken together, the agent's architecture mimics key cognitive system functions such as perception, memory, deliberation, and learning—within a procedural AI framework. Its generalization stems not from brute-force search but from intelligent transformation selection, structured by past performance and guided by heuristically informed array analysis.

### 1.2 Mapping to Cognitive System Layers

To contextualize the agent's design in terms of a cognitive architecture, it can be described through the layered model of cognition, as shown in Figure 1. At the base, my agent exhibits reactive behavior when applying direct transformations (like rotation or color changes) based on simple input features—this is the **Reaction** layer. As the agent gains exposure to training pairs and stores useful transformation results, this aligns with **Memory**, which it taps into during prediction. **Learning** is evident in its ability to adapt transformation priorities based on past success rates.

The crux of the agent resides in the **Deliberation** layer, where it actively reasons over the extracted features from the grid and infers the most probable solution paths. The agent currently lacks formal meta-cognition. A more mature implementation could incorporate true **Metacognition** by evaluating not just which transformation to choose but whether its overall approach should shift mid-task based on failure patterns. There is still a vibrant and active field of research implementing approaches to meet this layer so the hope is this will be solved at some point in the future and lead us into a new era of informed few-shot learning.

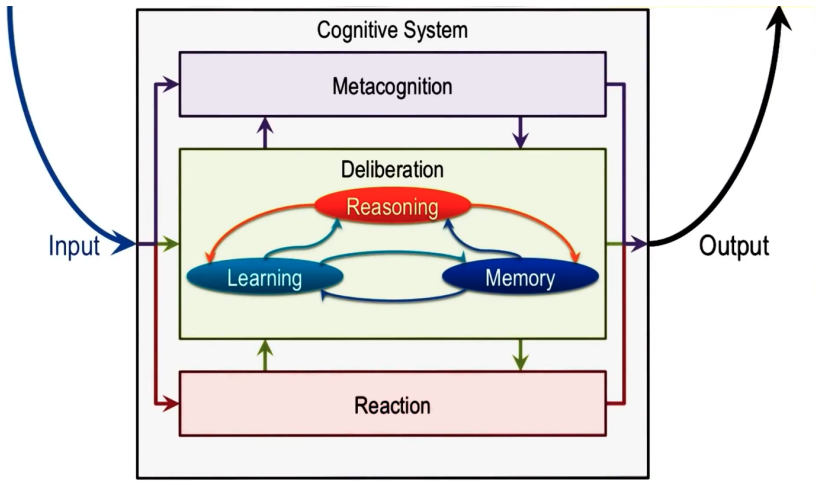


***Figure 1***—Cognitive system framework for agent design from the lectures. The agent maps closely to these layers, especially deliberation and memory.

## 2 AGENT PERFORMANCE

To evaluate the effectiveness of the proposed ARC-AGI agent, a comprehensive performance assessment was conducted across 96 visual reasoning problems distributed among three milestone categories: B, C, and D. These problem sets collectively represent increasing levels of difficulty and abstraction, thereby offering a diverse test set for the agent's generalization capabilities.

The final implementation achieved a score of 61 out of 96, indicating successful problem resolution in approximately 63.5% of the cases.

**Milestone Breakdown**

In Milestone B, which primarily consists of training-aligned problems, the agent exhibited high reliability, solving 15 out of 16 tasks. This performance underscores the agent's ability to extract and apply learned transformations from known data effectively.

In contrast, Milestone C posed moderately higher complexity. Here, the agent succeeded in solving 12 out of 16 problems, revealing a partial drop in generalization as the tasks diverged from direct training analogues to more abstract logic.

Milestone D, composed of the most abstract and cognitively demanding problems, presented significant challenges. The agent managed to solve only 6 out of 16 tasks. This result illustrates the current limitations of the architecture, particularly in

cases where multi-step reasoning, frame-detection, nested structure manipulation, or compositional generalization is required.

Overall, the agent's strengths lie in problems where visual transformations are cleanly separable, object-centric, or globally applicable. It showed consistent success in cases in these areas. However, its performance degrades in the presence of ambiguous mappings, overlapping color relationships, and higher-order structure reasoning. These patterns suggest promising researches for future improvement to reach more adaptive learning architectures in general. The results indicate that while the agent is strong at pattern recognition and transformation application, it still struggles with tasks requiring higher-order abstraction or unconventional reasoning through what we call "sideways thinking" where logic is correct a decent number of times but at times the agent needs to be able to toss out pre-conceptions of the idea of solving the problem based on previous transformations and fully come up with a new solution mechanism on-the-fly.

## 3 AGENT SUCCESSES

The agent performed particularly effectively in cases that aligned with its modular transformation pipeline and object-centric reasoning framework.

One category of successes involved problems requiring segmentation followed by some transformation. Making use of connected component labeling, the agent was able to detect and isolate objects within the input grid and independently apply context-appropriate transformations to each cluster. This per-object/cluster reasoning strategy was especially effective for tasks that required transformations based on the characteristics of the individual cluster. The modularity of the processing pipeline ensured that object-level results could be recombined into the final output with structural consistency. We use "5c0a986e" to showcase. The agent makes use of object detection to find the shapes after which it uses an diagonal extension heuristic with the learned color mappings to apply the tails of 1s and 2s. This process is showcased in Figure 2.

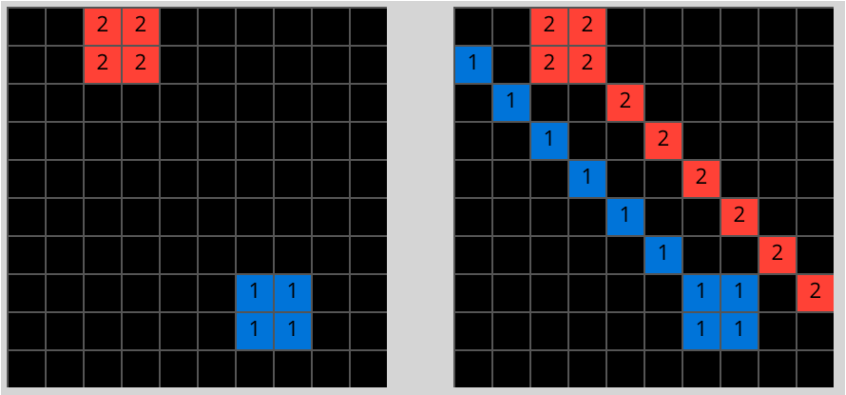

*Figure 2*—Solved tail extension heuristic.

A second area of strength was the agent's handling of enclosure-based problems. We choose the example of problem "7b6016b9" to showcase. Using the `is_shape_enclosed` routine, the agent successfully detected hollow regions surrounded by boundary pixels. We identified these by "talking a walk" around the border of the shape. Then the agent fills in the color using the `ndimage.binary_fill_holes` function. The agent then uses `NumPy.unique_counts` to conduct a color frequency analysis to choose the filler color. Lastly, the agent overlays the original values of the frame over the colored cells. This was able to be chained with other transformations to solve a decent number of problems. The visual process for this is showcased in Figure 3.

The agent also performed reliably on problems involving global grid transformations, such as reflections, rotations, and inversions. These transformations were prioritized within the `determine_transformation`. We made use of the vectorized built-in NumPy functions to run logical bitwise operators (OR, XOR, NAND, etc.) on the grid as well as grid sections based on splits decided by color barriers. We use "ce4f8723.json" to showcase this, since the logical operators are written into the NumPy library the agent need not do much to handle these cases. Mainly the operation for ce4f8723 requires the agent to make use of `split_sections` to split the input array. This is a built-in functionality of the agent that was implemented to split up an input array whenever the agent detects a horizontal or vertical colored divider line in the array. After which it is simply the matter of applying the logical operator between the 2 masks and applying color transformations based on the learned color mappings. The visual process is showcased in Figure 4

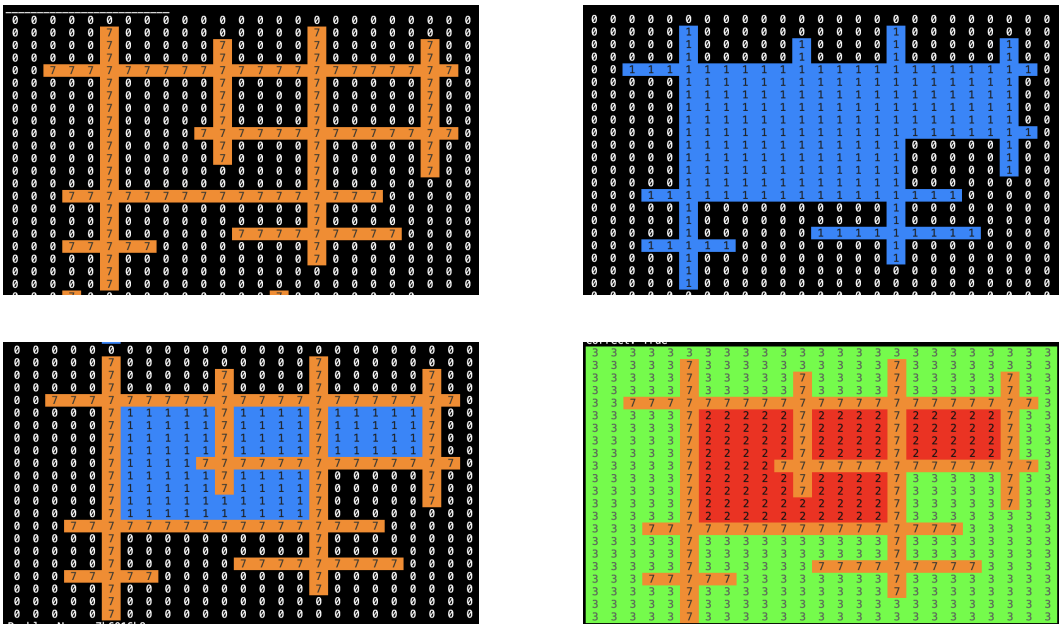

*Figure 3*—Solving process for 7b6016b9.json, starting top-left to bottom-right showcasing the object detection and enclosure checks.

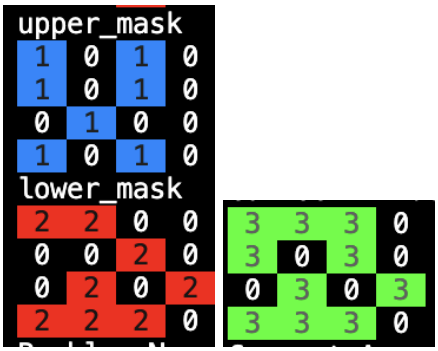


*Figure 4*—Solving process for ce4f8723.json, starting left to right showcasing the logical operators in action.

Collectively, these successful outcomes showcased the robustness of the agent's architecture when operating on the curated dataset. Its ability to combine object segmentation, transformation ranking, and heuristic evaluation enables scalable and interpretable reasoning over a diverse set of ARC tasks.

## 4 AGENT STRUGGLES

Despite its strengths, the agent encountered difficulties in certain complex scenarios. One challenging scenario involved multiple nested hollow shapes with un-

clear color boundaries. Here, the enclosure detection method mistakenly grouped nested shapes, leading to incorrect color assignments. This revealed a limitation in the heuristic used for filling enclosed areas.

Another area of struggle was ambiguous color mappings, particularly when input-output pairs contained overlapping colors without clear one-to-one relationships. The agent's assumption of direct color mappings caused inaccuracies, suggesting the need for more sophisticated color inference mechanisms. These limitations were especially apparent in Milestone D problems, where the tasks often required deeper reasoning or layered rule application.

## 5 AGENT DEVELOPMENT APPROACH

Throughout the project's development, there was an emphasis placed on creating a flexible and modular system. Initially, the agent focused heavily on global transformations and just solving the minimum required problems for each milestone. But this proved inadequate for obvious reasons. Recognizing this limitation, I integrated object segmentation and enclosure detection to address more nuanced challenges that I read through Team, 2024 and Wang, He, and Hariharan, 2021. On top of this a larger set of more robust and smartly chained transformations were added. Over time, heuristics for color mapping and noise removal were iteratively refined based on observed performance.

Right now the architecture is certainly best described as a never-ending expanding suite of heuristics. This was purposefully chosen to combat issues with the limitation of time constraints and computational resources placed from the grading mechanism. Were these both a non-issue, I would have left in the expensive chaining functionality I experimented with. To be more specific, it took the agent somewhere in the neighborhood of 43.26 minutes to take solve just 48 problems. considering the final size of the test set was 96 problems, it was deliberately decided to go down the path of pre-selecting the chains.

Additionally, I did look into trying to make processes faster through the use of caching and Multiprocessing and Asynchronous threads. The issue with this became that there were race conditions created due to the vectorized NumPy and SciPy functions used. That is to say that these functions were operationally as fast as they can be in this simple case. The results are shown in Figure 5

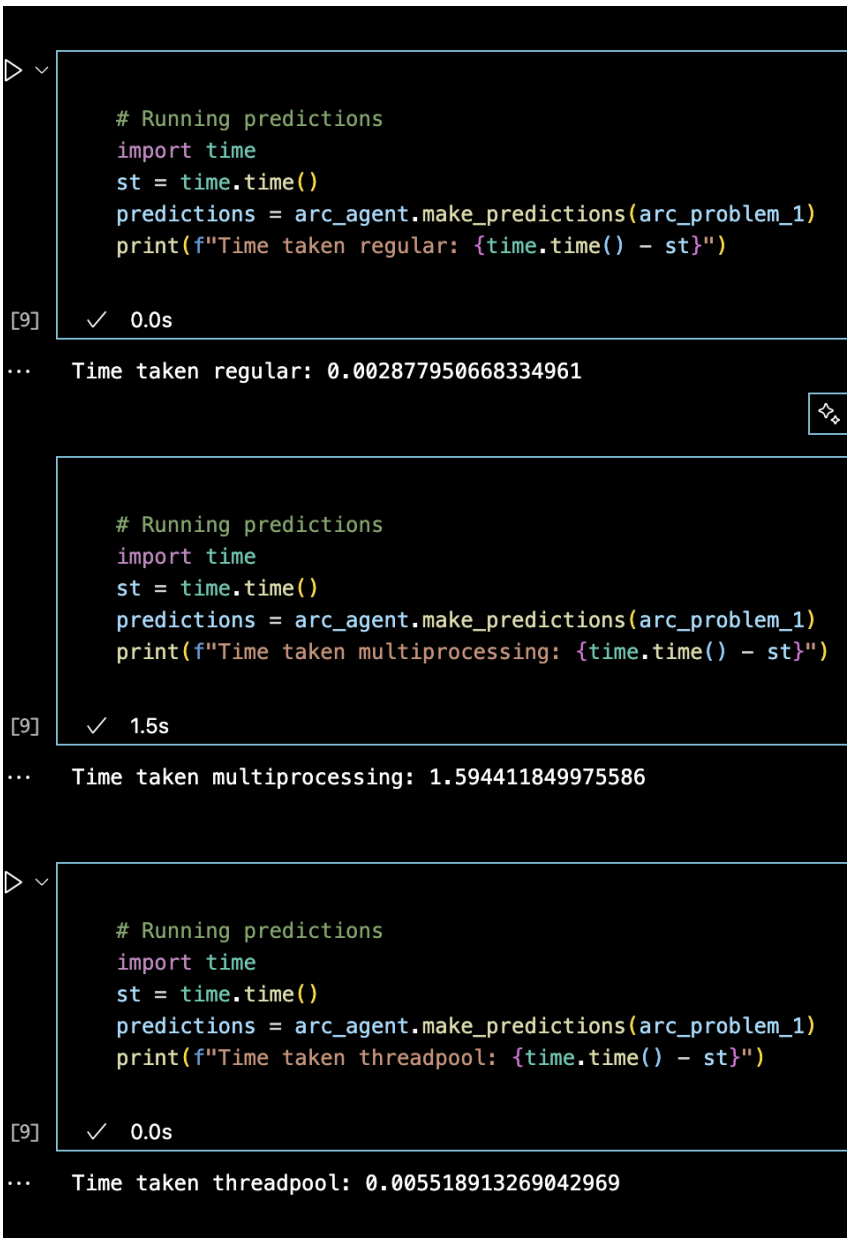


*Figure 5*—Results of the speed comparisions between regular, multiprocessing, and asyncio threaded approaches.

## 6 AGENT-HUMAN COMPARISON

The agent mirrors several aspects of human reasoning, particularly in how it breaks down complex problems into smaller, manageable parts. Like a human solver, it identifies patterns, applies learned transformations from prior examples, and uses object segmentation to break down the grid into meaningful units. This behavior

is the same as human *chunking*, where individuals process information in grouped segments to simplify cognitive load. Similarly, the agent's use of transformation ranking based on past success aligns loosely with human *episodic memory*, allowing it to recall and prioritize strategies that previously led to correct outcomes.

However, the agent falls short in areas where humans excel. Most notably in handling ambiguity, forming abstract concepts, and adapting on the fly. Humans can reason through unfamiliar tasks by drawing on *intuitive priors*, flexible analogies, and semantic cues, often hypothesizing potential rules without seeing exact matches in training examples. The agent, by contrast, relies on explicitly defined heuristics and lacks the ability to generalize beyond surface-level transformations. It does not reflect on failures in a meta-cognitive manner or change strategies mid-task. This limitation becomes particularly evident in problems requiring "sideways thinking," where prior patterns must be temporarily discarded in favor of novel approaches.

Ultimately, while the agent showcases similar cognitive parallels in perception, memory, and rule application, it lacks the *dynamic abstraction* and *self-directed learning* seen in humans. So it is close but not close enough.

## 7 CONCLUSION

Overall, the ARC-AGI agent showed solid performance across training and testing sets, passing 61 of 96 problems. It effectively applies transformations and heuristics to a wide range of visual reasoning tasks and performs especially well on problems requiring segmentation and consistent pattern rules. However, its limitations became clear when facing higher-level reasoning tasks or problems involving ambiguous/abstract mappings. These results show the challenges in building an agent that can mirror the incomprehensible flexibility and abstraction of human reasoning—but they also show a promising foundation for continued improvement.

## 8 REFERENCES


1. Chollet, François (2019). *Abstraction and Reasoning Corpus (ARC)*. https://github.com/fchollet/ARC. Accessed: 2025-04-21.
2. Harris, Charles R, Millman, K Jarrod, Walt, Stéfan J van der, Gommers, Ralf, Virtanen, Pauli, Cournapeau, David, Wieser, Eric, Taylor, Julian, Berg, Sebastian, Smith, Nathaniel J, et al. (2020). "Array programming with NumPy". In: *Nature* 585.7825, pp. 357–362. DOI: 10.1038/s41586-020-2649-2.
3. Virtanen, Pauli, Gommers, Ralf, Oliphant, Travis E., Haberland, Matt, Reddy, Tyler, Cournapeau, David, Burovski, Evgeni, Peterson, Pearu, Weckesser, Warren, Bright, Jonathan, Walt, Stéfan J. van der, Brett, Matthew, Wilson, Joshua, Millman, K. Jarrod, Mayorov, Nikolay, Nelson, Andrew R. J., Jones, Eric, Kern, Robert, Larson, Eric, Carey, C. J., Polat, Ilhan, Feng, Yu, Moore, Eric W., VanderPlas, Jake, Laxalde, Denis, Perktold, Josef, Cimrman, Robert, Henriksen, Ian, Quintero, E. A., Harris, Charles R., Archibald, Anne M., Ribeiro, Antõnio H., Pedregosa, Fabian, Mulbregt, Paul van, and Contributors, SciPy 1.0 (2020). "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". In: *Nature Methods* 17, pp. 261–272. DOI: 10.1038/s41592-019-0686-2.
4. Wang, Xinlei, He, Zhirong, and Hariharan, Bharath (2021). "The Surprising Effectiveness of Test-Time Training for Few-Shot Learning". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12235–12244. URL: https://openaccess.thecvf.com/content/CVPR2021/html/Wang_The_Surprising_Effectiveness_of_Test-Time_Training_for_Few-Shot_Learning_CVPR_2021_paper.html.
5. Team, ARC Prize (2024). *ARC Prize 2024: Technical Report*. https://arcprize.org/reports/arcprize2024.pdf. Accessed: 2025-04-21.